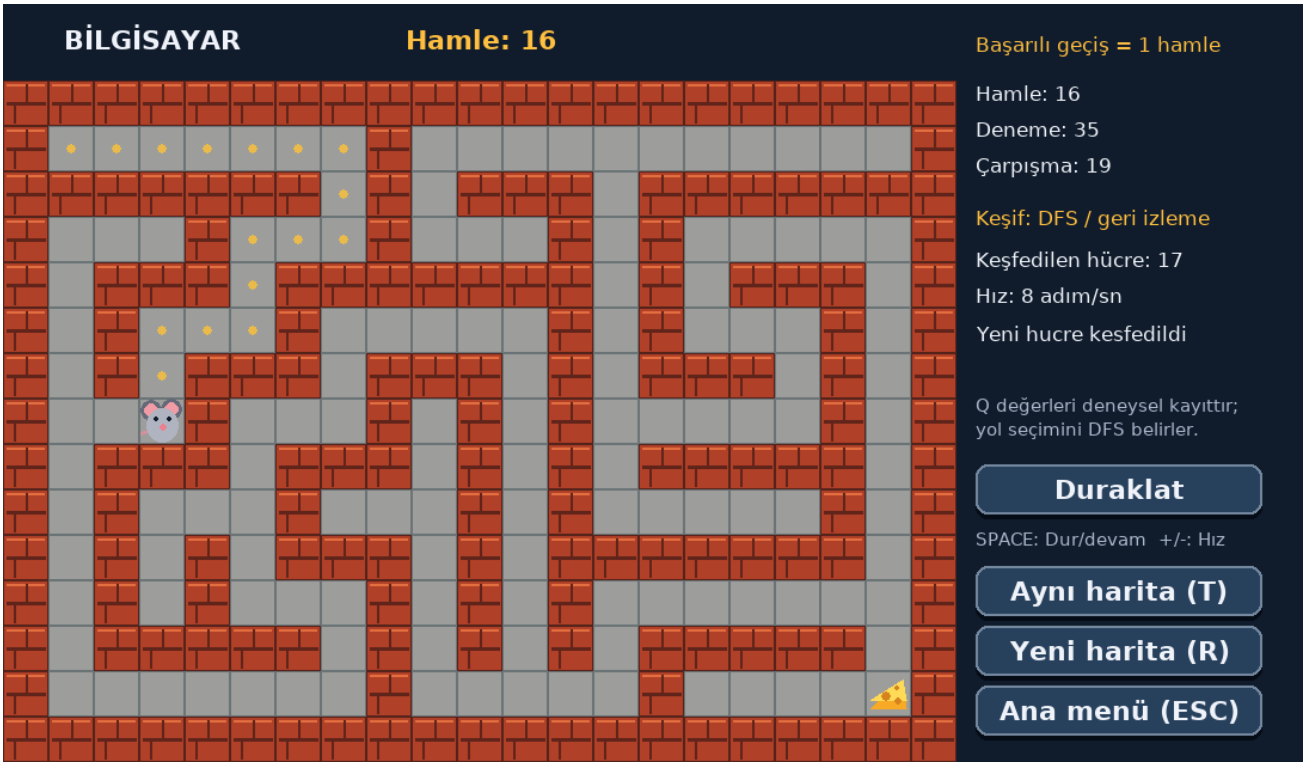
Labirent
Fare ve Peynir

Proje dokümantasyonu • Sınıf Çalışması (Labirent) - 4

Bahar Uz ve Haktan Tekinel

Python / Pygame / TCP • 11 Ekim 2026



Amaç ve kapsam

Kullanıcı veya bilgisayar bir labirentte peynire ulaşır. İki oyunculu modda aynı yerel ağdaki iki fare aynı haritada yarışır. Tüm modlarda hamle sayılır; deneme ve duvar çarpışması ayrıca gösterilir.

Ders isterleri: bilgisayar modu, manuel mod, iki kullanıcı için ağ üzerinden oyun, her modda hamle takibi, en az üç test senaryosu ve dokümantasyon. Bu belgede üç kabul senaryosu ve 15 otomatik testin sonucu bulunur.

Depo: github.com/Baharuz/labirent

Kurulum ve kullanım

Windows PowerShell

python -m venv .venv

.\.venv\Scripts\python.exe -m pip install -r requirements.txt

.\.venv\Scripts\python.exe src\main.py

Komutları proje kökünde çalıştırın. Python 3.10+ önerilir. Doğrulama Python 3.12.14 ve Pygame 2.6.1 ile yapılmıştır.

Oyun kontrolleri

Manuel: WASD / yön tuşları. R yeni harita; T aynı haritayı sıfır sayaçla yeniden başlatır.

Bilgisayar: Space dur/devam; +/- hızı 1-30 adım/sn aralığında değiştirir. Ayarlar ekranında keşif izi açılıp kapatılır.

Çok oyunculu: WASD / yön tuşları. Kazanma sonrası oda sahibi R ile yeni yarış başlatır.

Her mod: ESC ana menüye döner; pencere kapatılınca bağlantılar kapanır.

İki bilgisayarı bağlama

1. İki bilgisayarı aynı Wi-Fi/LAN ağına bağlayın. Birinci bilgisayarda Çok oyunculu > Oda kur seçin.

2. Oda sahibinde ipconfig çalıştırın. Aktif Wi-Fi/Ethernet bağlantısının IPv4 adresini bulun.

3. Diğer bilgisayarda Çok oyunculu ekranındaki adresi Backspace ile silip bu IPv4 adresini yazın. Odaya katıl seçin.

4. Harita ve hazır onayı eşitlenince yarış başlar. Gri fare P1, mavi fare P2’dir. İlk ulaşan kazanır.

127.0.0.1 yalnızca aynı bilgisayardaki iki pencereyi denemek içindir. Python’a özel ağ güvenlik duvarı izni gerekebilir; misafir Wi-Fi cihaz izolasyonu bağlantıyı engelleyebilir. TCP portu 50505’tir. İnternet port yönlendirmesi gerekmez.

Bağlantı koparsa oyun durur. ESC ile menüye dönüp yeni oda açın; yarım yarış kaldığı yerden devam ettirilmez.

Algoritma ve mimari

Labirent ve hareket

15 × 21 ızgarada rastgele geri izlemeli üretim kullanılır. Geçitler birbirine bağlıdır. Başlangıç (1,1), hedef (19,13) hücresidir. Testlerde seed parametresi aynı haritanın tekrar üretilmesini sağlar.

Hamle: Bitişik ve geçilebilir bir hücreye başarılı geçiştir. Duvara çarpınca hamle artmaz; deneme ve çarpışma artar. Geri dönüş hareketleri de sayılır. Her modda deneme = hamle + çarpışma. Kazanma sonrasında hareket durur.

Bilgisayarın peyniri bulması

Fare, seçtiği yönün sonucuna bakarak duvar ve geçit hafızası oluşturur. Denenmemiş komşulara ilerler; çıkmazda yol yığını üzerinden geri döner. Bu yaklaşım DFS (derinlik öncelikli arama) olarak adlandırılır. Hedefe uzaklık veya tüm harita üzerinden en kısa yol hesabı yapılmaz.

Mevcut rl_agent.py dosyasında Q-Learning değerleri deneysel olarak korunmuştur. Yol seçimini DFS belirler; bu sürüm için öğrenilmiş bir politikanın başarı artışı iddia edilmez. Aynı haritada yeniden başlatma keşif hafızasını ve Q tablosunu sıfırlar. Ders görselinde RL zorunlu değildir.

Ağ akışı

Oda sahibi haritayı init mesajıyla gönderir; istemci doğrulayıp ready yanıtını verir. Oda sahibi state gönderince yarış başlar. İstemci move mesajıyla yalnızca yön bildirir. Konum, hamle ve kazanan oda sahibi tarafından hesaplanır.

Her yarışın tur kimliği vardır; eski tura ait mesajlar işlenmez. TCP okuma/yazma ayrı iş parçacıklarındadır. Kuyruklar ve mesaj boyutu sınırlıdır. Bozuk veri veya bağlantı kopmasında hareket durur, uygulama menüye dönüşe izin verir.

Kodun görevleri

main.py: ekranlar ve olaylar; rendering.py: çizim; maze.py: üretim; player.py: hareket/sayaçlar; rl_agent.py: keşif; network.py: TCP; multiplayer.py: yarış protokolü; tests/test_game.py: doğrulama.

Test raporu ve teslim

Sonuç: 15 otomatik test başarılı, 0 başarısız. Ortam: Linux, Python 3.12.14, Pygame 2.6.1. Ekranlar SDL dummy sürücüsüyle; ağ gerçek TCP loopback bağlantısıyla doğrulandı.

Test komutu: python -m unittest discover -s tests -v

TC-01 / Manuel oyun

Adımlar: Bir duvara çarp, iki hücre ilerleyip hedefe ulaş, kazanma sonrasında tekrar hareket et.
Beklenen: 2 hamle, 3 deneme, 1 çarpışma; kazanma sonrası sayaçlar değişmez.
Sonuç: Otomatik test geçti.

TC-02 / Bilgisayar modu

Adımlar: 100 seed ile farklı labirent üret; bilgisayarı her birinde keşfe başlat.
Beklenen: Hedefi bulma ve tutarlı sayaçlar.
Sonuç: 100/100 labirentte hedef bulundu. Bağlantı ayrıca BFS ile doğrulandı.

TC-03 / İki oyunculu yarış

Adımlar: TCP ile bağlan, haritayı eşitle, istemciyi hedefe götür, yeni yarış başlat, bağlantıyı kes.
Beklenen: Aynı harita ve kazanan; ayrı ve eşitlenen sayaçlar; yeni turda sıfırlama; kopmada hareketin durması.
Sonuç: Aynı bilgisayardaki gerçek TCP bağlantısında geçti. İki fiziksel cihaz kontrolü henüz yapılmadı.

Ek kontroller ve sınırlar

Geçersiz yön/konum/mesaj, çok büyük mesaj, portun serbest kalması, iptal edilen bağlantı, küçük harita, başlangıç=hedef, aynı haritayı sıfırlama, duraklatma ve ekran çizimleri test edildi. Menü ve bilgisayar ekranları görsel olarak incelendi.

Teslim öncesi: İki ayrı bilgisayarda bir tam yarış, yeni tur ve bağlantı kopması denenmelidir. Wi-Fi ve güvenlik duvarı davranışı otomatik testlerle doğrulanmış sayılmaz.

Mail: Bu PDF ek olarak hazırdır. Alıcı bilgisi verilmediği için mail gönderilmemiştir.